

SCE221, Computer Networks
Assignment 1
Due: 11:59 PM, Oct 24th (Saturday), 2026
Total Score: 180 points
Instructor: Yiwen Shen
Ajou University, Korea

Name: _____, Student ID: _____

Submission Instructions:

- You can use a separate document/notes to complete your answers. Upload your answers by a PDF file in the corresponding assignment section of Ajou Bb. For programming questions, please submit your source code files and a README file that explains how you implemented the programs, as well as any extra functions you implemented.
- Your PDF file may be scanned from your handwritten answers (please ensure the scanned document is readable if you use a smartphone) or generated from an electronic document created in MS Word or on a tablet (e.g., iPad or Galaxy Tab).
- Remember to write down your name and student ID in the submitted PDF file.
- Please DO NOT directly copy any generated answers from any LLM-based tools into your solutions.

Note: For each question, please write down the calculation/analysis process. If NO proper process is given, the point for that question will be 0.

1 Basic Network Concepts

1. Suppose users share a 2 Mbps link. Also suppose each user transmits continuously at 1 Mbps when transmitting, but each user transmits only 20 percent of the time.
 - (a) (2 points) When circuit switching is used, how many users can be supported?
 - (b) (4 points) For the remainder of this problem, suppose packet switching is used. Why will there be essentially no queuing delay before the link if two or fewer users transmit at the same time? Why will there be a queuing delay if three users transmit at the same time?
 - (c) (2 points) Find the probability that a given user is transmitting.
 - (d) (4 points) Suppose now there are three users. Find the probability that at any given time, all three users are transmitting simultaneously. Find the fraction of time during which the queue grows.
2. Compare the time required for circuit switching and packet switching for the following conditions:
 - The destination is 3 hops away from the source (i.e., two routers in between).
 - The distance between any two adjacent nodes is 10 Km.
 - The message size is 1 Mega bits.
 - The bandwidth of all links is 50Kbps.
 - The maximum packet size is 100 K bits (You can ignore the size of the header).
 - The signal propagation speed is 10^6 m per second.
 - The circuit setup time is 10^{-2} second.
 - The processing time for routing decision at each node is negligible

where Mega bits = 10^6 bits and Kbps = 10^3 bits/sec.

- (a) (4 points) What is the data delivery time by *circuit switching*?
 - (b) (4 points) What is the data delivery time by *packet switching*?
3. A disadvantage of a broadcast subnet is the capacity wasted when multiple attempts to access the channel at the same time. Suppose that time is divided into discrete slots, with each of the n hosts attempting to transmit on the channel with probability p during each slot. That is, each station always has a packet to transmit.

- (a) (4 points) What fraction of the slots is wasted due to both collisions and idle?
- (b) (4 points) What is the probability for a transmitting station to be successful?
- (c) (4 points) What is the expected number of slots wasted (not including the last successful one) before a station successfully transmitted the current packet?

2 HTTP Lab

Before you run Wireshark, there are several things you should check in your network connection and browser configuration

- Make sure your browser program is not, by default, using the HTTP/3 protocol or the QUIC protocol. As of 2026, most popular browsers have adopted HTTP/3 and QUIC as defaults. A document that describes how to disable HTTP/3 AND QUIC as the defaults for your browser is here: <https://techysnoop.com/disable-quic-protocol-in-chrome-edge-firefox/>.
- Make sure then when you go to a URL, that the URL starts with “http://” rather than with “https://” as the latter case will encrypt frames.
- Make sure privacy and browser privacy settings are turned off.
- Make sure you clear your browser cache and browser history before you start.

Note: after you have completed the lab, you can re-enable HTTP/3 and QUIC as the defaults for your browser and other privacy settings.

Let’s begin our exploration of HTTP by downloading a very simple HTML file - one that is very short, and contains no embedded objects. Do the following:

1. Start up your web browser.
2. Start up the Wireshark packet sniffer, as described in the wireshark -lab slides (but don’t yet begin packet capture). Enter “http” (just the letters, not the quotation marks, and in lower case) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window. (We’re only interested in the HTTP protocol here, and don’t want to see the clutter of all captured packets).
3. Wait a bit more than one minute (we’ll see why shortly), and then begin Wireshark packet capture.
4. Enter the following to your browser:
`http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html`. Your browser should display the very simple, one-line HTML file.
5. Stop Wireshark packet capture.

Your Wireshark window should look similar to the window shown in Figure 1.

Stop Wireshark packet capture, and enter “http” (again, in lower case without the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.

4. For the basic HTTP GET/response interaction, answer the following by inspecting the HTTP messages in the packet trace of Wireshark.
 - (a) (4 points) Is your browser running HTTP version 1.0, 1.1, or 2? What version of HTTP is the server running? Please attach the screenshot of the packet trace of Wireshark where you find the information.
 - (b) (1 point) What languages, if any, does your browser indicate that it can accept?
 - (c) (2 points) What are the IP addresses of your computer and the gaia.cs.umass.edu server?
 - (d) (1 point) What status code does the server return to your browser?
 - (e) (1 point) When was the retrieved HTML file last modified at the server?
 - (f) (2 points) How many bytes of content are returned to your browser?
 - (g) (2 points) In the raw packet data, do you see any headers that are not displayed in the packet-listing window? If so, name one.

The HTTP CONDITIONAL GET/response interaction

- Start up your web browser, and make sure your browser’s cache is cleared, as discussed above.

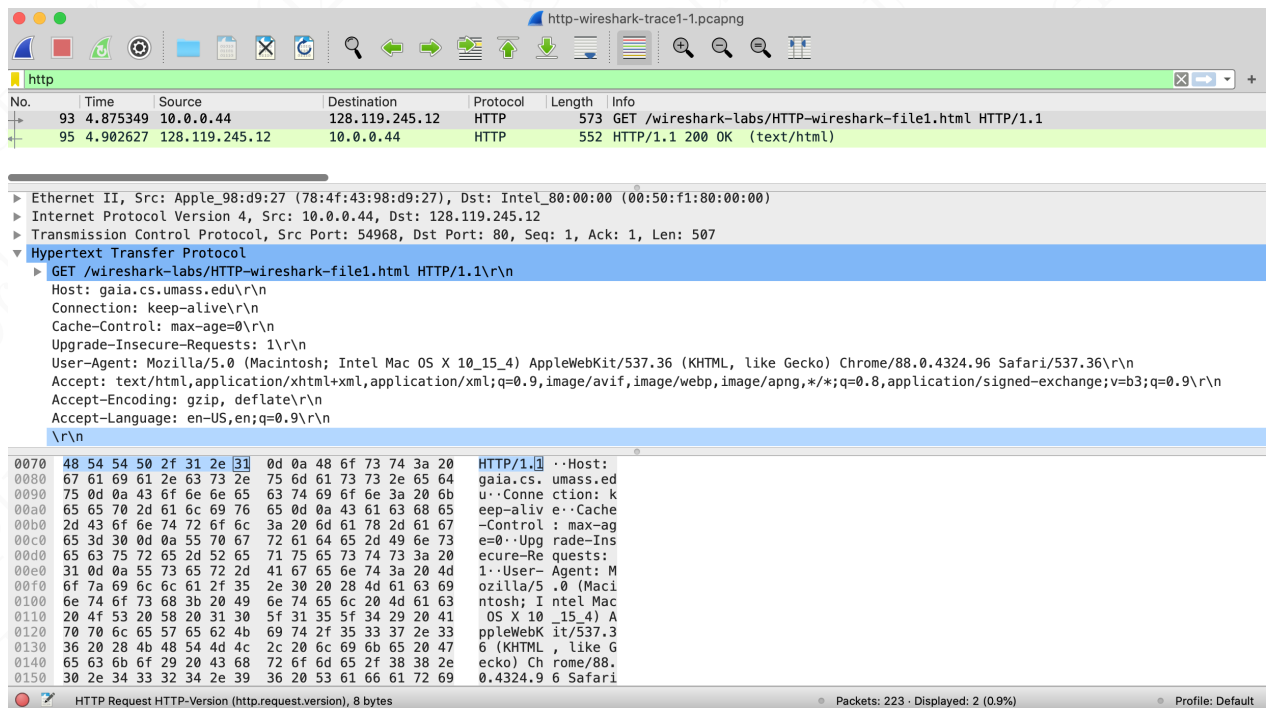


Figure 1: Wireshark window after capturing the HTTP GET/response interaction for a simple HTML file

- Start up the Wireshark packet sniffer
 - Enter the following URL into your browser
<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html> Your browser should display a very simple five-line HTML file.
 - Quickly enter the same URL into your browser again (or simply select the refresh button on your browser)
 - Stop Wireshark packet capture, and enter “http” (again, in lower case without the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.
5. For the HTTP conditional GET/response interaction, answer the following by inspecting both HTTP GET requests and the server responses.
- (a) (4 points) Does the first HTTP GET contain an If-Modified-Since header? Show screenshot of the packet trace of Wireshark where you find the information.
 - (b) (2 points) Did the server explicitly return the file contents in response to the first GET? How can you tell?
 - (c) (3 points) Does the second HTTP GET contain an If-Modified-Since header? If so, what information follows it?
 - (d) (3 points) What status code and phrase does the server return in response to the second GET? Did the server explicitly return the file contents? Explain.

Retrieving Long Documents

Let's next see what happens when we download a long HTML file. Do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
<http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html> Your browser should display a long HTML file, which is the text of the Bill of Rights.
- Stop Wireshark packet capture, and enter “http” (again, in lower case without the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.

In the packet-listing window, you should see your HTTP GET message, followed by a multiple-packet TCP response to your HTTP GET request. Make sure you your Wireshark display filter is cleared so that the multi-packet TCP response will be displayed in the packet listing.

In the case of our HTTP GET, the entity body in the response is the entire requested HTML file. In our case here, the HTML file is rather long, and at 4500 bytes is too large to fit in one TCP packet. The single HTTP response message is thus broken into several pieces by TCP, with each piece being contained within a separate TCP segment. In recent versions of Wireshark, Wireshark indicates each TCP segment as a separate packet, and the fact that the single HTTP response was fragmented across multiple TCP packets is indicated by the “TCP segment of a reassembled PDU” in the Info column of the Wireshark display.

6. For the long HTML document retrieval, answer the following by inspecting the packet trace.
 - (a) (4 points) How many HTTP GET request messages did your browser send? Which packet contains the GET for the Bill of Rights? Show screenshot of the packet trace of Wireshark where you find the information.
 - (b) (2 points) Which packet contains the status code and phrase associated with the response to the GET request?
 - (c) (2 points) What status code and phrase are in the response?
 - (d) (4 points) How many data-containing TCP segments carried the single HTTP response and the text of the Bill of Rights?

HTML Documents with Embedded Objects

Now that we’ve seen how Wireshark displays the captured packet traffic for large HTML files, we can look at what happens when your browser downloads a file with embedded objects, i.e., a file that includes other objects (in the example below, image files) that are stored on another server(s).

Do the following:

- Start up your web browser, and make sure your browser’s cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
`http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file4.html` Your browser should display a simple HTML file that contains two embedded images, one of which is stored on the gaia.cs.umass.edu server, and the other of which is stored on the www.cs.umass.edu server.
- Stop Wireshark packet capture, and enter “http” (again, in lower case without the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.

7. For the HTML document with embedded objects, answer the following by inspecting the packet trace.
 - (a) (6 points) How many HTTP GET request messages did your browser send, and to which Internet addresses were they sent? Show screenshot of the packet trace of Wireshark where you find the information.
 - (b) (2 points) Can you tell whether your browser downloaded the two images serially or in parallel? Explain.

HTTP Authentication

Finally, let’s try visiting a web site that is password-protected and examine the sequence of HTTP message exchanged for such a site. The URL `http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html` is password protected.

The username is “**wireshark-students**” (without the quotes), and the password is “**network**” (again, without the quotes).

So let’s access this “secure” password-protected site. Do the following:

- Start up your web browser, and make sure your browser’s cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
`http://gaia.cs.umass.edu/wireshark-labs/protected_pages/HTTP-wireshark-file5.html`
- When prompted, enter the username and password given above.

- Stop Wireshark packet capture, and enter “http” (again, in lower case without the quotation marks) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window.

Now let’s examine the Wireshark output. You might want to first read up on HTTP authentication by reviewing the easy-to-read material on “HTTP Access Authentication Framework” at https://medium.com/@code_and_chill/http-authentication-methods-and-strategies-to-protect-your-application-cda2ce9147a7 or <https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication>.

8. For the HTTP authentication exchange, answer the following by inspecting the HTTP messages in the packet trace.
- (a) (4 points) What status code and phrase does the server return in response to the initial HTTP GET? Show screenshot of the packet trace of Wireshark where you find the information.
 - (b) (2 points) What new field is included when your browser sends the HTTP GET for the second time?

The username (wireshark-students) and password (network) that you entered are encoded in the string of characters (d2lyZXNoYXJrLXN0dWlbnRzOm5ldHdvcm0=) following the “Authorization: Basic” header in the client’s HTTP GET message. While it may appear that your username and password are encrypted, they are simply encoded in a format known as Base64 format. The username and password are not encrypted! To see this, go to <http://www.motobit.com/util/base64-decoder-encoder.asp> and enter the base64-encoded string d2lyZXNoYXJrLXN0dWlbnRz and decode. Voila! You have translated from Base64 encoding to ASCII encoding, and thus should see your username! To view the password, enter the remainder of the string Om5ldHdvcm0= and press decode. Since anyone can download a tool like Wireshark and sniff packets (not just their own) passing by their network adaptor, and anyone can translate from Base64 to ASCII (you just did it!), it should be clear to you that simple passwords on WWW sites are not secure unless additional measures are taken.

3 DNS Lab

The nslookup command

The `nslookup` command can invoke the underlying DNS services to implement its functionality. The `nslookup` command is available in most Microsoft, Apple IOS, and Linux operating systems. To run `nslookup`, you just type the `nslookup` command on the command line in a DOS window, Mac IOS terminal window, or Linux shell.

In its most basic operation, `nslookup` allows the host running `nslookup` to query any specified DNS server for a DNS record. The queried DNS server can be a root DNS server, a top-level-domain (TLD) DNS server, an authoritative DNS server, or an intermediate DNS server (see the textbook for definitions of these terms). For example, `nslookup` can be used to retrieve a “Type=A” DNS record that maps a hostname (e.g., `www.nyu.edu`) to its IP address. To accomplish this task, `nslookup` sends a DNS query to the specified DNS server (or the default local DNS server for the host on which `nslookup` is run, if no specific DNS server is specified), receives a DNS response from that DNS server, and displays the result.

Let’s take `nslookup` out for a spin! We’ll first run `nslookup` on the Linux command line on Ajou University campus, where the local name server has an IP address 10.255.255.254. Let’s try `nslookup` in its simplest form:

In this example the `nslookup` command is given one argument, a hostname (`www.nyu.edu`). In words, this command is saying “please send me the IP address for the host `www.nyu.edu`.” As shown in the screenshot, the response from this command provides two pieces of information:

(1) the name and IP address of the DNS server that provides the answer – in this case the local DNS server at Ajou University;

and (2) the answer itself, which is the canonical host name and IP address of `www.nyu.edu`. You may have noticed that there are two name/address pairs provided for `www.nyu.edu`.

- The first (65.8.54.4, and 3 others) is an IPv4 address in the familiar-looking dotted decimal notation;
- the second (2600:9000:204d:1c00:1:f7e2:cb00:93a1, and 7 others) is a longer and more complicated looking IPv6 address.

We’ll learn about IPv4 and IPv6 and their two different addressing schemes later soon. For now, let’s just focus on our more comfortable (and common) IPv4 world.

Although the response came from the local DNS server (with IP address 10.255.255.254) at Ajou University, it is quite possible that this local DNS server iteratively contacted several other DNS servers to get the answer.

```

chris@ZenbookDuo: $ nslookup www.nyu.edu
Server:      10.255.255.254
Address:     10.255.255.254#53

Non-authoritative answer:
www.nyu.edu canonical name = d1q5ku5vnwkd2k.cloudfront.net.
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 65.8.54.4
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 65.8.54.23
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 65.8.54.83
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 65.8.54.71
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 2600:9000:204d:1c00:1:f7e2:cb00:93a1
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 2600:9000:204d:ba00:1:f7e2:cb00:93a1
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 2600:9000:204d:e600:1:f7e2:cb00:93a1
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 2600:9000:204d:ec00:1:f7e2:cb00:93a1
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 2600:9000:204d:f800:1:f7e2:cb00:93a1
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 2600:9000:204d:a200:1:f7e2:cb00:93a1
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 2600:9000:204d:a00:1:f7e2:cb00:93a1
Name:   d1q5ku5vnwkd2k.cloudfront.net
Address: 2600:9000:204d:5200:1:f7e2:cb00:93a1

```

Figure 2: the basic nslookup command

In addition to using nslookup to query for a DNS “Type=A” record, we can also use nslookup to nslookup to query for a “TYPE=NS” record, which returns the hostname (and its IP address) of an authoritative DNS server that knows how to obtain the IP addresses for hosts in the authoritative server’s domain.

```

chris@ZenbookDuo: $ nslookup -type=NS www.nyu.edu
Server:      10.255.255.254
Address:     10.255.255.254#53

Non-authoritative answer:
www.nyu.edu canonical name = d1q5ku5vnwkd2k.cloudfront.net.
d1q5ku5vnwkd2k.cloudfront.net nameserver = ns-1782.awsdns-30.co.uk.
d1q5ku5vnwkd2k.cloudfront.net nameserver = ns-45.awsdns-05.com.
d1q5ku5vnwkd2k.cloudfront.net nameserver = ns-744.awsdns-29.net.
d1q5ku5vnwkd2k.cloudfront.net nameserver = ns-1040.awsdns-02.org.

Authoritative answers can be found from:
ns-1040.awsdns-02.org internet address = 205.251.196.16
ns-1782.awsdns-30.co.uk internet address = 205.251.198.246
ns-45.awsdns-05.com internet address = 205.251.192.45
ns-744.awsdns-29.net internet address = 205.251.194.232
ns-1040.awsdns-02.org has AAAA address 2600:9000:5304:1000::1
ns-1782.awsdns-30.co.uk has AAAA address 2600:9000:5306:f600::1
ns-45.awsdns-05.com has AAAA address 2600:9000:5300:2d00::1
ns-744.awsdns-29.net has AAAA address 2600:9000:5302:e800::1

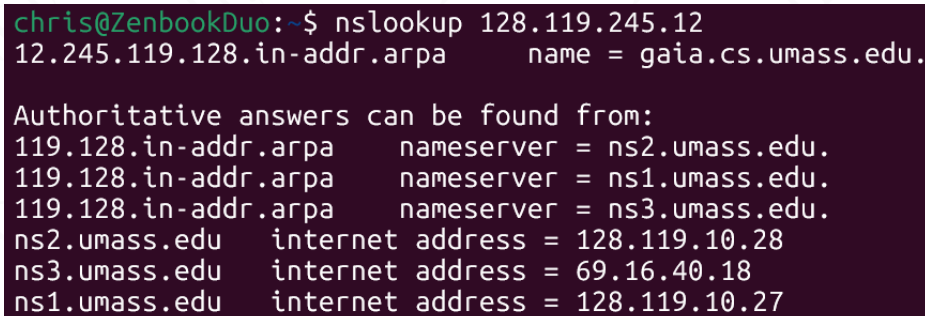
```

Figure 3: using nslookup to find the authoritative name servers for the nyu.edu domain

In the example in Figure 3, we’ve invoked nslookup with the option “-type=NS” and the domain “nyu.edu”. This causes nslookup to send a query for a type-NS record to the default local DNS server. In words, the query is saying, “please send me the host names of the authoritative DNS for nyu.edu”. (When the -type option is not used, nslookup uses the default, which is to query for type A records.) The answer, displayed in the above screenshot, first indicates the DNS server that is providing the answer (which is the default local Ajou University DNS server with address 10.255.255.254) along with three NYU DNS name servers. Each of these servers is indeed an authoritative DNS server for the hosts on the NYU campus. However, nslookup also indicates that the answer is “non-authoritative,” meaning that this answer came from the cache of some server rather than from an authoritative NYU DNS server. Finally, the answer also includes the IP addresses of the authoritative DNS servers at NYU. (Even though the type-NS query generated by nslookup did not explicitly ask for the IP addresses, the local DNS server returned these “for free” and nslookup displays the result.)

nslookup has a number of additional options beyond “-type=NS” that you might want to explore. Here’s a site with screenshots of ten popular nslookup uses: <https://www.cloudns.net/blog/10-most-used-nslookup-commands/> and here are the “man pages” for nslookup: <https://linux.die.net/man/1/nslookup>.

Lastly, we sometimes might be interested in discovering the name of the host associated with a given IP address, i.e., the reverse of the lookup shown in Figure 2 (where the host’s name was known/specified and the host’s IP address was returned). nslookup can also be used to perform this so-called “reverse DNS lookup.” In Figure 4, for example, we specify an IP address as the nslookup argument (128.119.245.12 in this example) and nslookup returns the host name with that address (gaia.cs.umass.edu in this example)



```
chris@ZenbookDuo:~$ nslookup 128.119.245.12
12.245.119.128.in-addr.arpa      name = gaia.cs.umass.edu.

Authoritative answers can be found from:
119.128.in-addr.arpa  nameserver = ns2.umass.edu.
119.128.in-addr.arpa  nameserver = ns1.umass.edu.
119.128.in-addr.arpa  nameserver = ns3.umass.edu.
ns2.umass.edu  internet address = 128.119.10.28
ns3.umass.edu  internet address = 69.16.40.18
ns1.umass.edu  internet address = 128.119.10.27
```

Figure 4: using nslookup to perform a “reverse DNS lookup”

Now that we’ve provided an overview of nslookup, it’s time for you to test drive it yourself. Do the following, **write down the results and attach screenshots from your own experiments**.

9. Using nslookup, answer the following questions based on the command output.
 - (a) (4 points) Use nslookup to find the IP address of `www.naver.com` and attach a screenshot of the output.
 - (b) (2 points) What is the IP address of the DNS server that provided the answer to the first query?
 - (c) (2 points) Did the answer to the first query come from an authoritative or non-authoritative server?
 - (d) (4 points) Use nslookup to find an authoritative name server for the `naver.com` domain. Give the first name returned. How would you find that server’s IP address?

The DNS cache on your computer

Most hosts (e.g., your personal computer) keep a cache of recently retrieved DNS records (sometimes called a DNS resolver cache), just like many Web browsers keep a cache of objects recently retrieved by HTTP. When DNS services need to be invoked by a host, that host will first check if the DNS record needed is resident in this host’s DNS cache; if the record is found, the host will not even bother to contact the local DNS server and will instead use this cached DNS record. A DNS record in a resolver cache will eventually timeout and be removed from the resolver cache, just as records cached in a local DNS server will timeout.

You can also explicitly clear the records in your DNS cache. There’s no harm in doing so – it will just mean that your computer will need to invoke the distributed DNS service next time it needs to use the DNS name resolution service, since it will find no records in the cache.

- On a Mac computer, you can enter the following command into a terminal window to clear your DNS resolver cache: `sudo killall -HUP mDNSResponder`
- On Windows, you can enter the following command at the command prompt: `ipconfig /flushdns`
- On Linux, you can enter the following command into a terminal window to clear your DNS resolver cache: `sudo systemd-resolve --flush-caches`
On v22.04 and later versions of Ubuntu Linux, enter: `sudo resolvectl flush-caches`

Now that we are familiar with nslookup and clearing the DNS resolver cache, we’re ready to get down to some serious business. Let’s first capture the DNS messages that are generated by ordinary Web-surfing activity.

- Clear the DNS cache in your host, as described above.
- Open your Web browser and clear your browser cache.
- Open Wireshark and enter `ip.addr == <your_IP_address>` into the display filter, where `<your_IP_address>` is the IPv4 address of your computer. With this filter, Wireshark will only display packets that either originate from, or are destined to, your host.

- Start packet capture in Wireshark.
- With your browser, visit the Web page: http://gaia.cs.umass.edu/kurose_ross/
- Stop packet capture.

Answer the following questions.

- For DNS tracing with Wireshark, answer the following questions using the corresponding packet traces.
 - (4 points) Find the first DNS query resolving gaia.cs.umass.edu. What is its packet number, and was it sent over UDP or TCP? Attach your screenshot of the packet trace of Wireshark.
 - (2 points) Find the response to that query. What is its packet number, and was it received over UDP or TCP?
 - (4 points) What is the destination port of the DNS query, and what is the source port of the DNS response?
 - (2 points) What IP address is the DNS query sent to?
 - (2 points) How many questions and answers are in the DNS query message?
 - (2 points) How many questions and answers are in the DNS response to the initial query?
 - (6 points) For the page at http://gaia.cs.umass.edu/kurose_ross/, identify the packet numbers of the initial HTTP GET, the DNS query and response used for that request, and the HTTP GET for http://gaia.cs.umass.edu/kurose_ross/header_graphic_book_8E_2.jpg. Also identify the DNS query associated with the image request and explain how DNS caching affects it.
- Now let's play with `nslookup` in a terminal.
 - Start packet capture.
 - Do an `nslookup` on www.cs.umass.edu
 - Stop packet capture.

You should get a trace that looks something like the Figure 5 in your Wireshark window. Let's look at the first type A query (which is packet number 19 in the figure below, and indicated by the "A" in the Info column for that packet.)

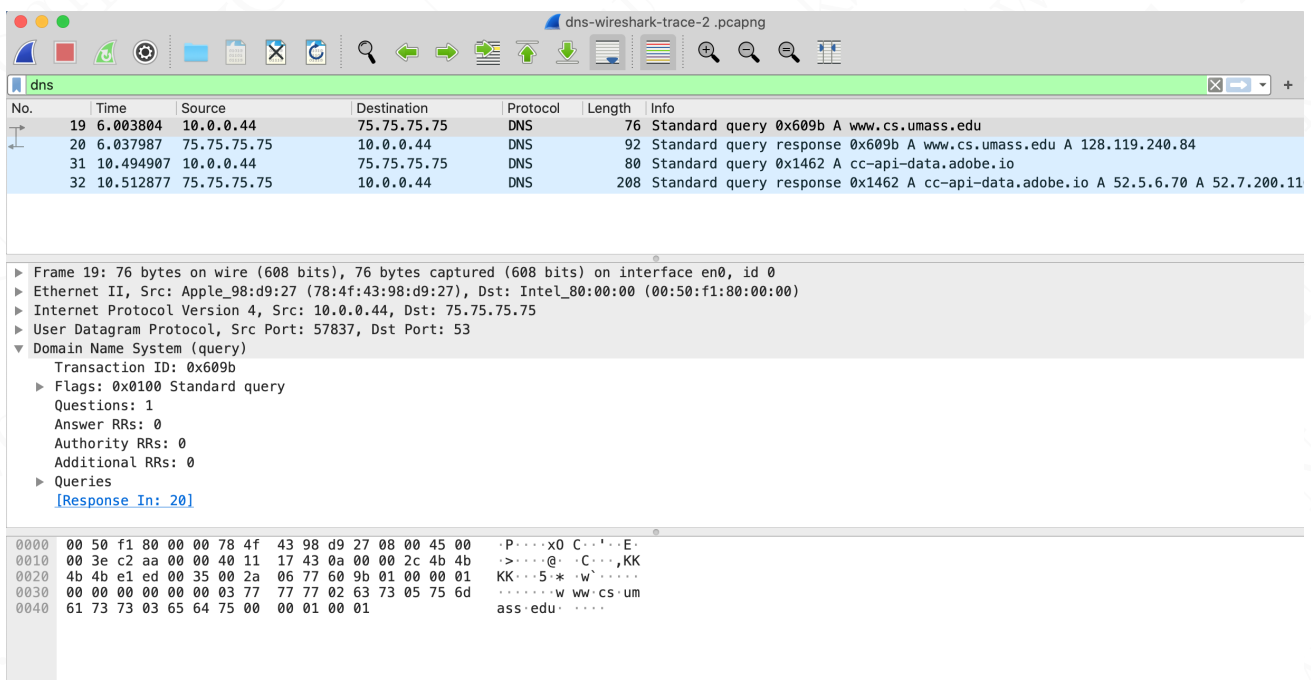


Figure 5: the first type A query for www.cs.umass.edu

- (4 points) For the `nslookup` query for www.cs.umass.edu, what is the destination port of the DNS query and the source port of its response? Show screenshot of the packet trace of Wireshark where you find the information.
- (2 points) To what IP address is that DNS query sent? Is this your default local DNS server?

- (c) (2 points) What type of DNS query is it, and does the query contain any answers?
- (d) (2 points) How many questions and answers are in the DNS response to that query?
12. Last, let's use nslookup to issue a command that will return a type NS DNS record, Enter the following command:

```
nslookup -type=NS umass.edu
```

and then answer the following questions

- (a) (2 points) For the nslookup query for umass.edu with type NS, to what IP address is the DNS query sent? Is this your default local DNS server?
- (b) (2 points) How many questions are in the DNS query, and does it contain any answers?
- (c) (4 points) How many answers are in the DNS response? What information do they contain, how many additional resource records are returned, and what information do those records provide?

Note: after you have completed the lab, you can re-enable HTTP/3 and QUIC as the defaults for your browser and other privacy settings.

4 Socket programming

13. (45 points) Building a Concurrent Multi-Threaded Chat Server (35+10 extra credit points)

The goal of this assignment is to design and implement a production-grade, concurrent Multi-User Chat Server using the standard BSD Sockets API and POSIX Threads (pthreads). Through this project, you will learn how to manage multiple asynchronous timelines, protect shared memory segments, handle unexpected network disconnections, and build an application-layer network protocol.

For this assignment, you need to have a linux environment with basic C programming tools (gcc, make, etc.) and a terminal. For Windows users, you can use WSL (Windows Subsystem for Linux) to run a linux environment on your Windows machine, or you can use a virtual machine (e.g., VirtualBox or VMware) with a linux distribution installed.

You will modify two separate programs by filling in the missing code sections. The two programs are:

1. `chat_server.c`: A multi-threaded daemon that listens for incoming TCP links, manages a global registry of active users, and safely broadcasts incoming messages across all connected channels.
2. `chat_client.c`: A lightweight client that connects to the server, allowing a user to choose a unique nickname and chat in real-time with other users across multiple terminal windows.

Functional Requirements:

1. The Server (`chat_server.c`)

- **Dynamic Connection Handling:** The server must listen on a port specified via a command-line argument (e.g., `./chat_server 8080`). It must accept up to 32 concurrent clients without blocking the main listener loop.
- **Worker Thread Allocation:** Each accepted client socket must be handed off to its own detached POSIX thread. The main thread must immediately return to listening for the next connection.
- **Shared Workspace Protection:** The server must track active client socket descriptors in a global array. Because multiple threads will read from and write to this array simultaneously, you must protect all array interactions using a mutual exclusion lock (`pthread_mutex_t`) to prevent race conditions.
- **Global Broadcasting:** When Client A transmits a message, the server must prepend the sender's nickname and forward the formatted message payload to all other active client file descriptors. It must not echo the message back to Client A.

2. The Client (`chat_client.c`)

- **Two-Way Concurrency:** The client must handle typing and receiving simultaneously. You should use a simple two-thread architecture:
 - Main Thread: Reads user inputs from stdin and writes them out to the server via `send()`
 - Reader Thread: Permanently blocks on `recv()`, dumping incoming server data to standard output (stdout) the moment it arrives.
- **Registration Protocol:** Upon first connecting, the client must prompt the user for a nickname (e.g., Alice). The client must send a special registration message to the server before regular chatting is permitted.

The makefile is provided to compile both programs. You can use the following command to build the server and client:

To build:

```
make all
```

To run and test:

```
./chat_server 8080
```

```
./chat_client 127.0.0.1 8080
```

You can open multiple terminal windows and run multiple clients to test the server's concurrent handling of messages.

Extra functions (10 points): You can implement the following extra functions to earn additional points. Each function is worth 5 points, for a total of 10 extra points.

- Implement an application-layer Command Parser. If a user types `/list`, the server responds directly to that specific client with a list of all currently connected nicknames.
- If a user types `/whisper <name> <message>`, the server routes the message exclusively to that specific user instead of a global broadcast. (To test this function, you can open three terminal windows, connect three clients, and have one client whisper a message to another client.)

Things to submit:

- Submit your source code files (`chat_server.c` and `chat_client.c`, and `chat_server_extra.c` with extra functions implemented) and a README file that explains how you implemented the programs, as well as any extra functions you implemented.
- Submit a screenshot and screen recording (less than 5 secs) of your server running with at least two clients connected and chatting.
- Submit a screenshot and screen recording (less than 5 secs) of your server's terminal showing the list of connected clients when a user types `/list`.
- Submit a screenshot and screen recording (less than 5 secs) of your server's terminal showing a whisper message being sent from one client to another.